

ALGOWINBET

Specifiche funzionali, algoritmiche, tecniche e di User Interface

Sistema adattivo per analisi probabilistica dei mercati sportivi, valutazione del valore atteso e ottimizzazione di combinazioni

DATA • PROBABILITÀ • MERCATO • INFORMAZIONE • OTTIMIZZAZIONE

Scopo del documento

Definire una specifica unica e implementabile per AlgoWinBet: un'applicazione che acquisisce dati sportivi e di mercato, ricostruisce lo stato informativo disponibile, stima probabilità calibrate per molteplici mercati, valuta la discrepanza tra prezzo e probabilità, analizza correlazioni e costruisce candidate slip rispettando vincoli e livelli di rischio impostabili dall'utente.

Il documento è volutamente indipendente da date, campagne commerciali e scadenze operative. Ogni componente è descritta come requisito, comportamento atteso, contratto dati o principio architetturale.

Sommario

1. Visione e principi fondanti

- 2. Obiettivi e perimetro del prodotto
- 3. Principi di progettazione dell'algoritmo

4. Architettura generale

- 5. Dominio applicativo e modello dati
- 6. Acquisizione e normalizzazione dei dati
- 7. Motore informativo e fonti
- 8. Motore temporale e snapshot
- 9. Feature engineering

10. Architettura dei modelli probabilistici

- 11. Calibrazione e gestione dell'incertezza
- 12. Modello del mercato e fair price
- 13. Motore di analisi dei mercati
- 14. Motore per combo e probabilità congiunte
- 15. Opportunity Engine e valutazione del valore
- 16. Correlation Engine
- 17. Optimizer e generazione della candidate slip
- 18. Risk Engine e bankroll
- 19. Event-driven re-ranking
- 20. Backtesting e paper betting
- 21. Explainability e audit

22. User Interface e User Experience

- 23. Specifica dettagliata delle schermate

- 24. Dashboard e mockup di riferimento

- 25. API e contratti applicativi

- 26. Job, code e orchestrazione

- 27. Persistenza e infrastruttura

- 28. Sicurezza, privacy e affidabilità

- 29. Testing e validazione

- 30. Osservabilità e qualità del modello

- 31. Struttura del repository

32. Criteri di accettazione del sistema

- 33. Comportamenti da evitare

- 34. Estensioni architetturali previste

- 35. Checklist finale di implementazione

- 36. Requisiti funzionali dettagliati

- 37. Core algorithmic flow e pseudocodice

- 38. Design system e componenti UI

- 39. User journey e stati dell'app

- 40. Contratti dati e schema JSON

- 41. Model lifecycle e retraining

- 42. Failure modes e graceful degradation

- 43. Scenario di utilizzo end-to-end

- 44. Principi di implementazione per Codex

- 45. Appendice: configurazioni e parametri

1. Visione e principi fondanti

AlgoWinBet è concepito come un motore di decisione probabilistica applicato ai mercati sportivi. Il suo compito centrale è trasformare un insieme eterogeneo di dati in stime probabilistiche calibrate, prezzi teorici, misure di edge, livelli di incertezza e candidate slip coerenti con i vincoli definiti dall'utente.

Idea guida

La domanda primaria del sistema è: “quale discrepanza misurabile esiste tra la probabilità stimata dal sistema e il prezzo osservabile sul mercato?”. La generazione della schedina è una conseguenza dell'analisi, non il punto di partenza.

1.1 Principi non negoziabili

- Forecasting e betting logic devono essere separati: il modello stima probabilità; il motore di mercato valuta prezzi; il selettore costruisce combinazioni.
- Il sistema deve poter produrre “NO BET” quando nessuna combinazione soddisfa i vincoli di qualità, valore e rischio.
- Ogni informazione deve essere legata a un timestamp di disponibilità per evitare contaminazione temporale e data leakage.
- Le probabilità devono essere calibrate e accompagnate da una misura di incertezza.
- Le combo devono essere valutate tramite probabilità congiunte o modelli di dipendenza, evitando la moltiplicazione cieca delle probabilità marginali.
- La qualità dei dati deve essere esplicita e deve entrare nelle decisioni del sistema.
- Il sistema deve conservare l'evidenza usata per ciascuna decisione in modo da poter ricostruire perché un'opportunità sia stata proposta o esclusa.
- Il comportamento del modello deve essere misurato con backtest temporali, paper betting, metriche probabilistiche e metriche di mercato.

1.2 Obiettivo concettuale

```

INPUT → informazioni disponibili + quote + calendario + mercati
      ↓
STATO → rappresentazione temporale e coerente della partita
      ↓
PROBABILITÀ → stima calibrata per evento e mercato
      ↓
PRICE → fair odds + confronto con quota osservata
      ↓
EDGE → valore teorico + incertezza + qualità dati
      ↓
PORTFOLIO → correlazioni + vincoli + ottimizzazione
      ↓
OUTPUT → candidate slip / watchlist / no bet / spiegazione
  
```

2. Obiettivi e perimetro del prodotto

2.1 Funzioni principali

- Analizzare un palinsesto selezionando sport, competizioni, periodo, mercati e vincoli.
- Acquisire un catalogo dinamico dei mercati disponibili dalla fonte configurata.
- Analizzare mercati principali, mercati alternativi, player props e combo quando disponibili.
- Integrare risultati storici, statistiche di squadra, statistiche individuali, quote, news e formazioni.
- Ricalcolare le stime quando cambia lo stato informativo della partita.

- Confrontare probabilità modello e prezzo di mercato.
- Individuare opportunità e classificare il livello di incertezza.
- Generare una o più candidate slip sotto vincoli espliciti.
- Consentire analisi manuale di una singola partita e dei suoi mercati.
- Conservare previsioni e risultati per backtesting e analisi degli errori.

2.2 Esclusioni dal core

- Automazione di versamenti o pagamenti.
- Automazione dell'invio di giocate verso piattaforme esterne.
- Scraping incontrollato senza gestione di policy, limiti o condizioni d'uso della fonte.
- Generazione di previsioni non tracciabili o prive di timestamp informativo.
- Adattamento della strategia in risposta emotiva a una singola perdita.

2.3 Profili d'uso

Profilo	Esigenza	Output principale
Esploratore	Vedere rapidamente le opportunità del palinsesto	Opportunity table + filtri
Analista	Comprendere perché il sistema assegna una probabilità	Match detail + explainability
Costruttore di slip	Raggiungere un intervallo di quota sotto vincoli	Optimizer + candidate slips
Backtester	Valutare il comportamento del sistema	Walk-forward + paper betting
Power user	Personalizzare mercati, soglie e rischio	Advanced settings + scenari

3. Principi di progettazione dell'algoritmo

3.1 Distribuzioni prima delle etichette

Per quanto possibile il sistema deve stimare distribuzioni degli esiti da cui derivare più mercati. Per il calcio, un modello dei gol attesi può produrre una distribuzione del punteggio, dalla quale derivare 1X2, Over/Under, BTTS, team totals e correct score.

3.2 Separazione tra modello informativo e modello di mercato

Il sistema deve mantenere separati almeno due concetti: un modello che usa informazioni sportive e contestuali e un modello che rappresenta il mercato attraverso le quote. Un ensemble può combinarli in uno strato successivo, consentendo di misurare quanto l'informazione indipendente dal prezzo aggiunga realmente valore.

3.3 Tempo come dimensione primaria

Ogni previsione è una funzione dello stato della partita e dell'istante di osservazione. La stessa partita può avere più previsioni valide in istanti diversi. Il record della previsione deve quindi contenere prediction_timestamp e information_cutoff.

3.4 Astensione come comportamento legittimo

Quando l'incertezza è elevata, l'offerta dati è incompleta o il mercato appare efficiente rispetto al modello, il sistema deve ridurre il ranking dell'opportunità o produrre NO BET. L'assenza di una proposta è un output valido e tracciabile.

4. Architettura generale



4.1 Stack tecnologico raccomandato

Area	Scelta consigliata	Responsabilità
Backend	Python + FastAPI	API REST, orchestration, validation
ML	Python + scikit-learn + LightGBM/XGBoost	modelli, calibration, evaluation
Probabilistic layer	SciPy + modelli Poisson/Bivariate/Dixon-Coles	distribuzioni e pricing
Database	PostgreSQL	entità applicative e risultati
Time-series / analytical	PostgreSQL + TimescaleDB oppure tabelle partizionate	snapshot quote e serie temporali
Cache / queue	Redis	cache, locks, job queue leggera
Worker	Celery/RQ/Arq	ingestion, feature jobs, recalculation
Frontend	Next.js + React + TypeScript	dashboard, filtri, visual analytics
UI	Tailwind CSS + component library	design system coerente
Charts	ECharts/Recharts	probabilità, line movement, calibration
Testing	pytest + Playwright	backend + end-to-end UI
Container	Docker + Compose per ambiente locale	riproducibilità e onboarding

Decisione architetturale

L'adozione di PostgreSQL come fonte di verità e di un layer analitico temporale evita di distribuire la logica del dominio in una moltitudine di storage. Il sistema deve privilegiare auditabilità e riproducibilità rispetto a una complessità infrastrutturale prematura.

5. Dominio applicativo e modello dati

5.1 Entità principali

Entità	Campi chiave	Note
Sport	id, name	Calcio, tennis, basket, ecc.
Competition	id, sport_id, country, name	Competizione/campionato
Team	id, competition_id, name, metadata	Entità stabile con alias
Player	id, team_id, position, metadata	Storico squadra gestito separatamente
Fixture	id, competition_id, home_id, away_id, kickoff	Evento sportivo canonico
MarketType	id, sport_id, code, category, schema	Catalogo normalizzato
MarketSelection	id, market_id, label, parameters	Esito/linea del mercato
OddsSnapshot	id, fixture_id, selection_id, book_id, timestamp, odds	Prezzo osservato
Source	id, type, name, reliability	Fonte e affidabilità
NewsItem	id, fixture/team/player, source, published_at, observed_at	Informazione strutturata
LineupSnapshot	fixture_id, observed_at, status, players, formation	Probabile/ufficiale
FeatureVector	fixture_id, cutoff, schema_version, payload	Feature usate dal modello
Prediction	selection_id, cutoff, model_version, p, interval	Probabilità e incertezza
Opportunity	selection_id, cutoff, odds, fair_odds, edge, ev, confidence	Valutazione del mercato
Correlation	selection_a, selection_b, score, method	Dipendenza stimata
CandidateSlip	id, constraints, selections, aggregate_metrics	Output optimizer
PaperBet	slip_id, created_at, odds, stake, result	Simulazione senza esecuzione esterna
Settlement	selection_id/slip_id, outcome, settled_at	Esito reale
BankrollSnapshot	user_id, timestamp, balance, exposure	Per analisi di rischio
AuditEvent	type, actor, timestamp, payload_hash	Tracciabilità

5.2 Requisiti di identificazione

- Gli identificativi interni devono essere UUID o equivalenti stabili.
- I nomi mostrati a UI non devono essere usati come chiavi primarie.
- Fixture e player devono avere un mapping verso gli identificativi dei provider.
- Le alias map devono essere versionate per gestire variazioni di denominazione.
- Le entità devono conservare provenance per sapere da quale fonte derivano.

6. Acquisizione e normalizzazione dei dati

6.1 Adapter pattern

Ogni provider deve essere implementato tramite un adapter con contratto comune. L'applicazione non deve conoscere direttamente il formato JSON specifico del provider.

```
ProviderAdapter
├── list_competitions()
├── list_fixtures()
├── list_markets(fixture_id)
└── get_odds(fixture_id)
```

```

└─ get_statistics(fixture_id)
└─ get_team_news(team_id)
└─ get_player_news(player_id)
└─ get_lineup(fixture_id)

```

6.2 Normalizzazione

Il normalizzatore converte dati eterogenei in un modello canonico. Ogni record deve conservare provider_name, provider_event_id, source_timestamp e normalized_at.

6.3 Gestione dei mercati dinamici

Il catalogo dei mercati deve essere trattato come configurazione dinamica. Per ogni mercato devono essere definiti sport, categoria, tipo di evento, parametri, eventuali outcome mutually exclusive, regole per il pricing e schema di visualizzazione.

Codice canonico	Categoria	Esempio parametro	Tipo
MATCH_1X2	Match	none	mutually exclusive
TOTAL_GOALS	Goals	line=2.5	over/under
BTTS	Goals	none	yes/no
TEAM_TOTAL	Team	team=HOME,line=1.5	over/under
PLAYER_GOAL	Player	player_id	binary
PLAYER_SHOTS	Player	player_id,line=2.5	over/under
COMBO_SGP	Combo	legs=[...]	joint/composite

7. Motore informativo e fonti

7.1 Gerarchia delle fonti

Livello	Tipologia	Uso
A	Fonte ufficiale del club/competizione	formazioni, comunicati, convocazioni, squalifiche
B	Provider sportivo strutturato	fixture, statistiche, quote, lineups
C	Media sportivo affidabile	contesto, dichiarazioni, news
D	Fonti secondarie	segnali aggiuntivi a bassa confidenza
E	Rumor/UGC	solo se configurato esplicitamente; mai trattato come fatto

7.2 News pipeline

1. Acquisire il contenuto con timestamp di pubblicazione e di osservazione.
2. Identificare entità coinvolte: fixture, team, player, coach.
3. Classificare la tipologia: injury, suspension, lineup, tactical, weather, statement, transfer, motivation, schedule, other.
4. Assegnare un confidence source_score in base alla fonte e alla completezza del dato.
5. Produrre un evento strutturato e conservarne il riferimento alla fonte originale.
6. Inviare l'evento al feature engine per il ricalcolo delle variabili interessate.
7. Innescare il re-ranking solo per i mercati potenzialmente sensibili a quell'informazione.

7.3 Estrazione da testo

L'LLM può essere utilizzato come strato di parsing e classificazione, non come oracolo delle probabilità.

L'output deve essere validato tramite schema Pydantic e regole deterministicamente verificabili.

```
{
  "event_type": "PLAYER_STATUS",
  "player_id": "...",
  "status": "OUT",
  "confidence": 0.96,
  "source_level": "A",
  "effective_from": "source timestamp",
  "evidence_ref": "source id"
}
```

8. Motore temporale e snapshot

8.1 Principio di information cutoff

Ogni modello deve ricevere esclusivamente informazioni osservabili entro un cutoff. Un record di previsione deve contenere prediction_timestamp, information_cutoff, odds_as_of, news_as_of e lineup_as_of.

8.2 Stati informativi consigliati

Stato	Contenuto
LONG_HORIZON	stagione, forza relativa, rating, storico
PRE_MATCH	forma, calendario, infortuni, news, mercato
LINEUP_CONFIRMED	XI ufficiale, modulo, panchina, ruoli
LAST_UPDATE	ultime variazioni informative prima del kickoff

8.3 Immutabilità degli snapshot

Uno snapshot storico non deve essere sovrascritto. Se una fonte corregge un dato, il sistema salva un nuovo record con provenance e relation_to_previous. Questo requisito è indispensabile per riprodurre un backtest.

9. Feature engineering

9.1 Feature di squadra

- Forza offensiva e difensiva normalizzata per competizione.
- Home/away split.
- Forma recente con pesi temporali.
- Expected goals e shot quality.
- Possesso, tiri, tiri in porta, corner e carte dove disponibili.
- Riposo tra partite e congestione calendario.
- Qualità media degli avversari affrontati.
- Trend di performance e stabilità del modulo.

9.2 Feature individuali

- Minuti attesi.
- Starting probability.
- Goal/assist rate per 90.
- Shots e shots on target per 90.

- Card rate.
- Ruolo e posizione.
- Responsabilità su rigori, punizioni e corner.
- Synergy/compatibility con compagni rilevanti.

9.3 Feature delle formazioni

Feature	Significato
lineup_confirmed	1 se XI ufficiale disponibile
formation	modulo canonico
formation_shift	distanza dal modulo atteso
key_absence_count	assenze di giocatori ad alto impatto
attack_quality	qualità offensiva dell'XI
defense_quality	qualità difensiva dell'XI
bench_quality	qualità dei sostituti disponibili
role_integrity	coerenza tra giocatori e ruoli attesi

9.4 Feature di mercato

- Quota corrente.
- Quota di apertura se disponibile.
- Movimento di quota.
- Dispersione tra bookmaker.
- Closing line storica quando disponibile.
- Overround stimato.
- Liquidità proxy quando disponibile.
- Tempo al kickoff.

10. Architettura dei modelli probabilistici

10.1 Layer 1 — modello strutturale

Per il calcio, il layer strutturale deve produrre intensità di gol e/o distribuzioni di punteggio tramite modelli Poisson, Bivariate Poisson o Dixon-Coles. Questo layer fornisce una base interpretabile e coerente tra più mercati.

10.2 Layer 2 — modelli tabellari

LightGBM/XGBoost o modelli equivalenti possono apprendere relazioni non lineari tra feature sportive, individuali, di mercato e temporali. Il modello deve essere addestrato con split temporali e feature versionate.

10.3 Layer 3 — ensemble

```
p_final = w_struct * p_struct + w_ml * p_ml + w_market * p_market
vincoli: w_i >= 0, sum(w_i) = 1
ottimizzazione dei pesi su validation temporale
calibrazione applicata dopo l'ensemble o nel modo previsto dalla pipeline
```

10.4 Modelli per market family

Market family	Modello consigliato	Output
1X2 / DNB	goal model + ML residual	probabilità esiti

Market family	Modello consigliato	Output
Over/Under goals	goal distribution	P(total > line)
BTTs	goal distribution + classifier residual	P(Y/N)
Correct score	joint goal distribution	matrice P(home,away)
Player goals	minutes + player scoring rate	P(score >= 1)
Player shots	count model / hurdle model	P(over line)
Cards	count model + referee/team features	P(over line)
Corners	count model + style/volume features	P(over line)
Combo/SGP	joint model / copula / simulation	P(all legs)

11. Calibrazione e gestione dell'incertezza

11.1 Calibration layer

La probabilità deve essere calibrata separatamente per market family, quando il volume dati lo consente. Metodi possibili: isotonic regression, Platt scaling, beta calibration o temperature scaling, selezionati tramite validation temporale.

11.2 Intervalli di incertezza

Ogni stima deve poter esporre un intervallo o un uncertainty score. L'intervallo può derivare da bootstrap, ensemble disagreement, conformal prediction o analogo metodo coerente con il tipo di modello.

11.3 Data Quality Score

Componente	Esempi di segnali
Completeness	quote presenti, statistiche disponibili, lineup disponibile
Freshness	recenza dei dati rispetto al cutoff
Source reliability	livello fonte
Consistency	concordanza tra provider
Model coverage	mercato già validato dal modello

Il sistema calcola un data_quality_score e lo utilizza per abbassare la confidence quando il contesto informativo è incompleto.

12. Modello del mercato e fair price

12.1 Probabilità implicita

```
p_raw = 1 / decimal_odds
fair_odds_model = 1 / p_model
```

12.2 Overround

Per mercati mutuamente esclusivi, la somma delle probabilità implicite permette di stimare il margine complessivo del book.

```
overround = sum_i(1 / odds_i) - 1
p_market_i = (1 / odds_i) / sum_j(1 / odds_j)
```

12.3 Value

```
EV(selection) = p_model * odds - 1
```

```
edge = p_model - p_market
```

L'app deve mostrare sia edge sia EV, evitando di trattarli come sinonimi. Per mercati non comparabili o combo complesse, la normalizzazione deve seguire una funzione specifica per market type.

13. Motore di analisi dei mercati

13.1 Pipeline

1. Scarica il catalogo dei mercati disponibili.
2. Normalizza mercato e selezioni.
3. Recupera quote per bookmaker/fonte.
4. Calcola price quality, overround e dispersione.
5. Seleziona il modello appropriato.
6. Calcola p_model e intervallo.
7. Calcola fair odds, edge, EV.
8. Applica data quality e uncertainty penalties.
9. Calcola market confidence e opportunity status.

13.2 Stati delle opportunità

Stato	Significato
STRONG	edge/EV consistente, incertezza contenuta, dati solidi
CANDIDATE	interessante ma con uno o più vincoli da verificare
WATCH	potenziale, richiede nuove informazioni
NEUTRAL	nessuna discrepanza rilevante
AVOID	rischio, dati o dipendenza sfavorevoli
INVALID	dati insufficienti o inconsistenti

14. Motore per combo e probabilità congiunte

14.1 Tipi di combo

- Combo tra partite diverse.
- Multiple cross-match.
- Same Game Parlay.
- Combo con player props.
- Combo tra mercato di squadra e mercato individuale.

14.2 Principio di pricing

Per una combo, il sistema deve calcolare la probabilità congiunta. Se la quota combo è fornita direttamente dalla piattaforma, il prezzo osservato della combo deve essere confrontato con il fair price congiunto del modello. Il prodotto delle quote delle singole gambe può essere usato come riferimento diagnostico e mai come sostituto del pricing della combo.

14.3 Approcci implementabili

Approccio	Uso	Complessità
Monte Carlo	pricing flessibile di molte leg	media

Approccio	Uso	Complessità
Copula / dependence model	dipendenze esplicite	alta
Bayesian network	relazioni causali interpretabili	alta
Joint neural model	molti mercati correlati	molto alta
Closed-form specifica	mercati con relazione semplice	bassa

14.4 Monte Carlo

```
for simulation in N:
    sample match state
    sample goals / player events / cards / corners
    evaluate every leg
P_joint = successful_simulations / N
```

15. Opportunity Engine e valutazione del valore

15.1 Opportunity score

Si definisce uno score interno, non mostrato come “voto assoluto”, che combina EV, edge, uncertainty, data quality, market movement, model agreement e correlation exposure.

```
opportunity_score =
    a * normalized_ev
  + b * normalized_edge
  + c * calibration_quality
  + d * data_quality
  + e * model_agreement
  - f * uncertainty
  - g * correlation_penalty
```

15.2 Regole di eleggibilità

- Soglia minima di probabilità opzionale.
- Soglia minima di EV opzionale.
- Soglia massima di uncertainty.
- Mercati esclusi esplicitamente dall’utente.
- Numero massimo di selezioni per slip.
- Limite di esposizione per team/partita/competizione.

16. Correlation Engine

16.1 Correlazione intra-match

Il motore deve stimare la dipendenza tra leg. Esempi: risultato + total goals, player goal + team win, shots + goals, cards + game state.

16.2 Correlazione cross-match

Tra partite diverse la dipendenza è generalmente inferiore, ma può emergere da mercati comuni, eventi di competizione o fattori esterni. Il motore deve quindi mantenere almeno un modello di correlazione conservativo e configurabile.

16.3 Matrice

```
C[i][j] = dependence(selection_i, selection_j)
```

```
C[i][i] = 0
penalty = aggregate(C[i][j])
```

17. Optimizer e generazione della candidate slip

17.1 Input utente

Parametro	Esempio di valore	Effetto
Quota minima	intervallo configurato	vincolo hard
Quota massima	intervallo configurato	vincolo hard
Mercati	1X2, O/U, BTTS, player props, combo	insieme candidati
Competizioni	selezione utente	insieme eventi
P minima	soglia configurabile	filtro
Max leg	numero configurabile	vincolo
Correlation limit	soglia configurabile	vincolo
Risk profile	Conservativo/Equilibrato/Dinamico	funzione obiettivo

17.2 Strategie di ricerca

- Beam search per mantenere un set ristretto di candidati.
- Branch and bound quando i vincoli lo rendono conveniente.
- Mixed Integer Optimization per problemi con vincoli forti.
- Monte Carlo candidate sampling per grandi spazi combinatori.

17.3 Funzione obiettivo

```
objective(S) =
  w1 * portfolio_ev(S)
+ w2 * joint_probability(S)
+ w3 * diversification(S)
- w4 * total_uncertainty(S)
- w5 * correlation_penalty(S)
- w6 * model_disagreement(S)
```

17.4 Output multiplo

L'optimizer deve poter restituire un piccolo set di candidate slip diverse tra loro, evitando di presentare sei varianti quasi identiche. La diversità deve essere misurata tramite overlap di selezioni e concentrazione sullo stesso match/team.

18. Risk Engine e bankroll

18.1 Separazione dei layer

```
Prediction → Value → Selection → Risk → Stake
```

Il bankroll non deve influenzare la probabilità stimata. Influenza esclusivamente l'esposizione e la gestione del rischio.

18.2 Regole di esposizione

- Limite per singola slip.
- Limite aggregato per partita.

- Limite aggregato per squadra.
- Limite per market family.
- Limite per giornata/competizione.
- Cap dell'esposizione totale.

18.3 Stake engine opzionale

Per gli utenti che desiderano simulare la dimensione della giocata, è previsto uno stake engine separato. Può includere flat stake, percentuale del bankroll o fractional Kelly con cap. Il sistema deve sempre permettere un tetto massimo configurabile e una modalità paper-only.

Regola di stabilità

Il motore di stake non deve aumentare automaticamente l'esposizione per recuperare perdite precedenti. Gli stake sono una conseguenza delle regole configurate, delle opportunità e dei limiti di rischio, non del risultato emotivo di una singola sessione.

19. Event-driven re-ranking

19.1 Eventi che possono innescare un ricalcolo

- Nuovo infortunio confermato.
- Squalifica o disponibilità aggiornata.
- Cambio allenatore o modulo.
- Formazione ufficiale.
- Variazione significativa della quota.
- Nuova fonte ufficiale.
- Cambio delle condizioni contestuali rilevanti.

19.2 Impact routing

Ogni tipo di evento deve avere una mappa dei mercati sensibili. Una formazione ufficiale può influire fortemente su marcatori, tiri e risultato; una notizia su un difensore può influire soprattutto su goal totals, BTTS e clean sheet.

Evento	Mercati da ricalcolare con priorità alta	Mercati da monitorare
Attaccante OUT	player goals, shots, team total	1X2, O/U, BTTS
Portiere OUT	1X2, BTTS, goals	correct score
Formazione ufficiale	player props, goals, shots	1X2, totals
Modulo inatteso	1X2, totals, BTTS	corners/cards
Quota move	value/EV, market confidence	optimizer
Meteo rilevante	goals, corners, shots	cards

20. Backtesting e paper betting

20.1 Walk-forward validation

Il backtest deve simulare il comportamento del sistema come se fosse eseguito nel passato: training solo su dati precedenti, test su un periodo successivo, aggiornamento progressivo. Nessuna feature può includere informazione osservata dopo il cutoff simulato.

20.2 Paper betting

Ogni candidate slip può essere trasformata in paper bet. Il record conserva la decisione esatta presa dal sistema, la quota vista, la probabilità stimata e tutte le feature rilevanti.

20.3 Metriche probabilistiche

Metrica	Scopo
Brier Score	accuratezza delle probabilità binarie
Log Loss	penalizzazione delle probabilità troppo sicure
Calibration curve	coerenza tra p stimata e frequenza osservata
ECE	errore medio di calibrazione
Sharpness	capacità di produrre probabilità informative
Coverage interval	qualità degli intervalli di incertezza

20.4 Metriche di mercato

Metrica	Scopo
EV realizzato	coerenza tra edge stimato e ritorno simulato
Closing line movement	misura indiretta della qualità del prezzo intercettato
Hit rate	descrittivo; mai sufficiente da solo
ROI paper	valutazione aggregata della strategia simulata
Drawdown	stabilità del percorso dei risultati
Turnover	numero e frequenza delle decisioni

21. Explainability e audit

21.1 Spiegazione utente

Ogni opportunità deve avere una spiegazione composta da evidenze strutturate. Il testo può essere generato da un LLM a partire da un payload deterministico, così che il linguaggio sia variabile mentre i numeri restano vincolati al record sorgente.

21.2 Schema di spiegazione

```
WHY RECOMMENDED
├── positive_factors[]
├── negative_factors[]
├── lineup_effect
├── market_effect
├── model_consensus
├── uncertainty
├── data_quality
└── what_would_change_it[]
```

21.3 Counterfactual analysis

Il sistema deve poter simulare scenari: “senza lineup ufficiale”, “senza movimento quota”, “assenza del giocatore X”, “modello senza market features”. L’obiettivo è mostrare quali fattori determinano realmente la proposta.

22. User Interface e User Experience

22.1 Obiettivo UX

L'interfaccia deve permettere a un utente non tecnico di passare da “voglio analizzare questo palinsesto” a “capisco quali opportunità il sistema ha trovato” con il minimo numero di passaggi. La UI deve privilegiare gerarchia visiva, densità informativa controllata e spiegazioni progressive.

22.2 Direzione visuale

- Dark theme come default.
- Blu notte profondo come base.
- Verde/teal come colore primario di azione e valore positivo.
- Rosso limitato a rischio, warning e valore negativo.
- Card con angoli arrotondati e bordi sottili.
- Numeri di probabilità, quota ed EV molto evidenti.
- Iconografia coerente per mercato, formazione, news e rischio.
- Layout desktop a tre aree: sidebar, contenuto centrale, pannello laterale.

22.3 Regole UX

- Ogni numero importante deve avere contesto.
- EV, probability ed edge devono essere sempre distinguibili.
- Le informazioni tecniche avanzate devono essere espandibili, non nascoste.
- L'utente deve poter passare dalla candidate slip alla singola partita in un clic.
- La stessa terminologia deve essere mantenuta in tutta l'app.
- Un cambio di filtri deve aggiornare chiaramente lo stato dell'analisi.

23. Specifica dettagliata delle schermate

23.1 Home / Dashboard

Area	Componenti	Comportamento
Header	profilo, notifiche, stato dati	accesso rapido a impostazioni e qualità fonte
Filter bar	quota target, periodo, competizioni	filtri globali persistenti
Hero	stato analisi, conteggio partite/mercati	mostra output sintetico
Candidate slip	lista selezioni, quota, p, EV, status	click apre match detail
Why this slip	motivazioni sintetiche	espandibile
Probability panel	probabilità aggregata + range	tooltip metodologico
Market panel	numero mercati analizzati	accesso all'explorer
Bankroll	budget, exposure, saldo simulato	modulo separato
Quick match search	ricerca squadra/campionato	accesso diretto alla partita

23.2 Ricerca Opportunità

Schermata tabellare ad alta densità per esplorare l'intero opportunity set. Filtri laterali per sport, competizione, market family, quota, probabilità, EV, confidence, data quality, stato e disponibilità della lineup.

23.3 Palinsesto completo

Vista calendario con tutte le partite filtrate. Ogni riga mostra fixture, ora, stato lineup, quota sintetica, opportunità top e indicatore di cambiamento recente.

23.4 Analisi mercati

Explorer dedicato che mostra tutti i market type disponibili per una fixture. Ogni mercato può essere ordinato per EV, edge, probabilità, quota, confidence e movement.

23.5 Match Detail

Schermata centrale dell'applicazione: overview, statistiche, formazioni, mercati, news, modelli, timeline, dipendenze e scenari.

23.6 Backtest

Ambiente analitico per confrontare modelli e strategie. Deve essere possibile selezionare una famiglia di mercati, un modello, una finestra temporale e una configurazione di optimizer, visualizzando metriche e curva cumulata.

24. Dashboard e mockup di riferimento

Il mockup seguente costituisce il riferimento visuale per la direzione del prodotto. Il branding mostrato nell'immagine è puramente dimostrativo; nell'implementazione il brand deve essere "AlgoWinBet". Il layout evidenzia il principio chiave: l'utente vede subito la sintesi della candidate slip, mentre i pannelli laterali permettono di accedere a analisi di partita, mercati completi e controllo del rischio.



Figura 1 — Mockup concettuale della dashboard AlgoWinBet

24.1 Lettura del mockup

Elemento del mockup	Traduzione funzionale
Barra filtri superiore	parametri globali: quota, periodo, competizioni, comando analisi
Hero “analisi completata”	riassunto del risultato dell’Opportunity Engine
Candidate slip centrale	output dell’Optimizer con metriche per leg
Perché questa schedina?	Explainability della selezione complessiva
Distribuzione probabilità	sintesi della probabilità aggregata e del range
Andamento nel tempo	serie delle quote e stato dell’opportunità
Analisi rapida	accesso diretto a fixture specifiche
Tutti i mercati disponibili	shortcut al Market Explorer
Gestione bankroll	rischio e budget configurati
Match detail inferiore	tab per probabilità, statistiche, formazioni, mercati e news

24.2 Evoluzione UI prevista

- Sostituire il placeholder di brand con identità AlgoWinBet.
- Rendere responsivi tutti i pannelli.
- Aggiungere status badge: dati aggiornati, lineup ufficiale, news nuove, quote cambiate.
- Aggiungere interazioni hover per evidenziare la relazione tra quota, fair odds ed EV.
- Consentire drill-down dalla slip alla fonte informativa che ha contribuito alla decisione.

25. API e contratti applicativi

25.1 Endpoint principali

Metodo	Endpoint concettuale	Funzione
GET	/fixtures	elenco fixture filtrabile
GET	/fixtures/{id}	dettaglio fixture
GET	/fixtures/{id}/markets	mercati disponibili
GET	/fixtures/{id}/analysis	analisi probabilistica
GET	/fixtures/{id}/timeline	timeline informativa
GET	/opportunities	opportunity set
POST	/analyze	avvia analisi con filtri utente
POST	/optimize-slip	genera candidate slip
GET	/slips/{id}	dettaglio candidate slip
POST	/paper-bets	crea paper bet
GET	/backtest	query risultati backtest
GET	/models	stato/versioni modelli
GET	/sources/health	stato fonti

25.2 Contratto di analisi

```
{
  "filters": {
    "competitions": ["..."],
    "markets": ["..."],
    "odds_range": [2.0, 15.0],
    "min_probability": 0.60,
    "max_legs": 8
  },
  "risk_profile": "balanced",
  "information_cutoff": "server-side",
  "output": {"include_explanations": true}
```

}

26. Job, code e orchestrazione

26.1 Job principali

- fixture_sync_job
- odds_sync_job
- stats_sync_job
- news_sync_job
- lineup_sync_job
- feature_build_job
- prediction_job
- calibration_job
- opportunity_rank_job
- optimizer_job
- settlement_job
- backtest_job
- model_monitor_job

26.2 Event bus logico

```
ODDS_UPDATED → MARKET_REVALUE  
LINEUP_UPDATED → FEATURE_REFRESH → PREDICTION_REFRESH  
NEWS_CLASSIFIED → IMPACT_ROUTING → SELECTIVE_RECALCULATION  
FIXTURE_SETTLED → PAPER_BET_SETTLEMENT → METRIC_REFRESH
```

26.3 Idempotenza

Ogni job deve essere idempotente. Lo stesso evento può essere ricevuto più volte senza creare duplicati o alterare la decisione storica. Utilizzare event_id, deduplication key e transaction boundary.

27. Persistenza e infrastruttura

27.1 Tabelle essenziali

- fixtures
- teams
- players
- competitions
- market_types
- market_selections
- odds_snapshots
- sources
- news_items
- lineup_snapshots
- feature_vectors
- predictions
- opportunities

- correlations
- candidate_slips
- paper_bets
- settlements
- bankroll_snapshots
- model_versions
- audit_events

27.2 Versioning

Feature schema, model artifact, calibration artifact, market normalization rules e optimizer configuration devono avere versioni indipendenti. Una Prediction deve poter ricostruire esattamente tutte le versioni usate.

27.3 Storage policy

I raw payload delle fonti devono essere conservati in forma append-only o in un object store compatibile, con hash per integrità. Il database conserva il normalized state e i riferimenti ai payload originali.

28. Sicurezza, privacy e affidabilità

- Segreti API soltanto lato server.
- Rotazione e segregazione delle credenziali dei provider.
- Rate limiting per endpoint costosi.
- Audit log delle modifiche a configurazioni e modelli.
- Cifratura dei dati personali e del bankroll associato all'account.
- Backup e restore verificabili.
- Health checks per ogni provider.
- Circuit breaker e fallback quando una fonte è indisponibile.
- Timeout e retry con exponential backoff.

28.1 Degrado controllato

Quando una fonte critica non è disponibile, l'app deve comunicare chiaramente la riduzione della qualità dati e può degradare alcune funzioni. Ad esempio, la sezione formazioni può restare in stato "stale" senza permettere al modello di assumere che il dato sia aggiornato.

29. Testing e validazione

29.1 Unit test

- Conversione quote → probabilità.
- Overround correction.
- Calcolo EV/edge.
- Feature computation.
- Normalization dei mercati.
- Correlazioni note in casi sintetici.
- Pricing di combo in fixture di test.
- Constraints dell'optimizer.

29.2 Integration test

- Provider adapter → normalized database.
- News parser → structured event.
- Lineup parser → lineup snapshot.
- Event → selective recalculation.
- Optimizer → candidate slip.
- Settlement → paper metrics.

29.3 Backtest integrity tests

- Verificare che feature future non entrino nel cutoff.
- Verificare che quote usate siano quelle osservabili.
- Verificare che lineup ufficiali compaiano solo dopo il loro observation timestamp.
- Verificare la riproducibilità della stessa prediction da snapshot immutabili.

30. Osservabilità e qualità del modello

30.1 Metriche di sistema

- latency ingestion
- latency prediction
- provider availability
- queue depth
- prediction throughput
- cache hit rate
- database query latency

30.2 Metriche ML

- Brier/LogLoss per market family
- calibration drift
- feature drift
- prediction drift
- model disagreement
- coverage degli intervalli
- performance stratificata per competizione/mercato

30.3 Data drift

Il sistema deve monitorare se distribuzione delle feature e distribuzione degli outcome cambiano in modo significativo. Un drift importante deve produrre un flag di “model review required”.

31. Struttura del repository

```
algowinbet/
├── apps/
│   ├── api/
│   └── web/
├── packages/
│   ├── domain/
│   ├── providers/
│   └── normalization/
```

```

— features/
— models/
— calibration/
— market_engine/
— correlation/
— optimizer/
— risk/
— explainability/
— evaluation/
— workers/
— migrations/
— tests/
— notebooks/
— configs/
— docker/
— docs/

```

31.1 Regola di separazione

Il codice di dominio non deve dipendere dalla UI. I provider non devono contaminare il dominio con i propri schemi. L'optimizer deve ricevere un set canonico di candidate selections e restituire un risultato serializzabile.

32. Criteri di accettazione del sistema

Area	Criterio di accettazione
Dati	un fixture può essere ricostruito da snapshot e provenance
Mercati	un nuovo market type può essere aggiunto tramite adapter/config senza riscrivere la UI core
Probabilità	ogni prediction conserva modello, calibrazione, cutoff e intervallo
Value	fair odds, edge ed EV sono riproducibili
Combo	la probabilità congiunta è esplicitamente distinta dal prodotto delle marginali
Lineup	una lineup ufficiale innesca il ricalcolo dei mercati sensibili
News	una news strutturata ha source, timestamp, confidence ed effetto tracciabile
Optimizer	le candidate slip rispettano tutti i vincoli hard
No Bet	il sistema può restituire zero candidate slip
Explainability	ogni proposta ha motivazioni e fattori negativi
Backtest	una prediction storica può essere rigenerata dal cutoff originale
UI	l'utente può passare da dashboard → slip → partita → mercato → fonte
Audit	le modifiche a configurazioni e modelli sono registrate

33. Comportamenti da evitare

- Presentare una probabilità come certezza.
- Usare accuracy come metrica unica.
- Moltiplicare probabilità marginali per prezzare automaticamente combo correlate.
- Mescolare informazioni post-evento nel training o nel backtest.
- Usare rumor come fatti senza evidenza e provenance.
- Cambiare il modello soltanto dopo una sequenza di risultati negativi.
- Generare sempre una slip per soddisfare la richiesta dell'utente.
- Nascondere la presenza di dati mancanti o stale.
- Mescolare gestione bankroll e previsione.

- Mostrare un “voto” complessivo che mascheri le metriche che lo compongono.

34. Estensioni architetturali previste

- Nuovi sport tramite sport-specific adapters.
- Nuove market family tramite MarketType registry.
- Multi-bookmaker price comparison.
- Scoring live con motore separato dal pre-match.
- Personalizzazione delle preferenze senza modificare le probabilità di base.
- Modelli gerarchici bayesiani per competizioni con pochi dati.
- Feature learning su sequenze temporali.
- Portfolio optimizer multi-slip.
- Sistema di alert quando una opportunità supera una soglia configurata.
- Knowledge graph di squadre, giocatori, ruoli e relazioni informative.

35. Checklist finale di implementazione

35.1 Fondazioni

- Definire dominio canonico.
- Implementare provider adapters.
- Costruire persistence e provenance.
- Implementare market registry.
- Implementare snapshot temporali.

35.2 Algoritmo

- Implementare baseline probabilistiche.
- Implementare modelli ML.
- Implementare ensemble.
- Implementare calibration.
- Implementare uncertainty.
- Implementare market pricing.
- Implementare opportunity engine.
- Implementare correlation engine.
- Implementare optimizer.

35.3 Informazione dinamica

- Implementare news ingestion.
- Implementare structured events.
- Implementare lineup ingestion.
- Implementare impact routing.
- Implementare event-driven recalculation.

35.4 Valutazione

- Implementare walk-forward backtest.
- Implementare paper betting.
- Implementare settlement.

- Implementare model monitoring.
- Implementare data drift monitoring.

35.5 UI

- Realizzare design system.
- Implementare dashboard del mockup.
- Implementare opportunity explorer.
- Implementare match detail.
- Implementare market explorer.
- Implementare optimizer workflow.
- Implementare backtest.
- Implementare bankroll e paper mode.

36. Requisiti funzionali dettagliati

36.1 Ingestion e palinsesto

ID	Requisito	Comportamento atteso
FR-001	Sincronizzazione fixture	Importare fixture, competizione, squadre, kickoff e stato.
FR-002	Catalogo mercati	Rilevare dinamicamente tutti i market type esposti dalla fonte.
FR-003	Quote multi-fonte	Conservare quote per provider, bookmaker e timestamp.
FR-004	Deduplica	Unire duplicati tramite chiavi provider e matching canonico.
FR-005	Stato fixture	Gestire scheduled, postponed, cancelled, live, finished e unknown.

36.2 Analisi

ID	Requisito	Comportamento atteso
FR-010	Analisi massiva	Valutare tutte le fixture che soddisfano i filtri.
FR-011	Analisi completa mercati	Valutare ogni mercato supportato e disponibile.
FR-012	Analisi singola fixture	Mostrare probabilità, fair price, EV, edge, uncertainty e motivazioni.
FR-013	Ricalcolo selettivo	Ricalcolare soltanto market family impattate da un evento.
FR-014	Confronto modelli	Esporre consenso/disaccordo tra modelli.

36.3 Optimizer

ID	Requisito	Comportamento atteso
FR-020	Quota target	Generare combinazioni entro un intervallo di quota configurato.
FR-021	Vincolo probabilità	Scartare leg che violano la soglia configurata.
FR-022	Vincolo correlazione	Imporre un limite alla dipendenza aggregata.

ID	Requisito	Comportamento atteso
FR-023	Diversificazione	Evitare candidate slip quasi identiche.
FR-024	No Bet	Restituire esito esplicito quando non esistono candidate eleggibili.

36.4 Informazione

ID	Requisito	Comportamento atteso
FR-030	News parser	Estrarre eventi strutturati da fonti configurate.
FR-031	Provenance	Collegare ogni informazione a fonte e timestamp.
FR-032	Lineup states	Distinguere expected, probable, confirmed e stale.
FR-033	Impact routing	Associare ogni tipo di news ai mercati sensibili.

36.5 UX

ID	Requisito	Comportamento atteso
FR-040	Filter persistence	Mantenere filtri tra dashboard, explorer e optimizer.
FR-041	Drill-down	Dashboard → slip → fixture → market → evidence.
FR-042	Explainaibility	Mostrare fattori positivi, negativi e incertezza.
FR-043	Data freshness	Mostrare lo stato di aggiornamento dei dati.

37. Core algorithmic flow e pseudocodice

37.1 Analisi completa del palinsesto

```
def analyze_opportunity_set(request):
    universe = fixture_service.query(request.filters)
    for fixture in universe:
        snapshot = state_builder.build(fixture, request.information_cutoff)
        feature_set = feature_engine.build(snapshot)
        market_catalog = market_service.available_markets(fixture)
        for market in market_catalog:
            if not model_registry.supports(market):
                continue
            prediction = predictor.predict(feature_set, market)
            calibrated = calibrator.transform(prediction)
            pricing = market_pricer.evaluate(calibrated, market)
            quality = quality_engine.score(snapshot, market)
            opportunity = opportunity_engine.score(pricing, quality)
            store(opportunity)
    return optimizer.generate(request.constraints)
```

37.2 Ricalcolo event-driven

```
def handle_information_event(event):
    impact = impact_router.resolve(event.type)
    snapshot = state_builder.refresh(event.fixture_id)
    features = feature_engine.patch(snapshot, impact.features)
```

```

affected = market_registry.expand(impact.market_families)
predictions = predictor.batch_predict(features, affected)
calibrated = calibrator.batch_transform(predictions)
opportunities = opportunity_engine.revalue(calibrated)
correlation_engine.refresh(event.fixture_id, affected)
optimizer.refresh_watchlists(event.fixture_id, opportunities)

```

37.3 Generazione candidate slip

```

def optimize(candidates, constraints):
    filtered = hard_filter(candidates, constraints)
    beam = seed_candidates(filtered)
    while beam and not stopping_condition(beam):
        expanded = expand_one_leg(beam, filtered)
        feasible = [s for s in expanded if satisfies_hard_constraints(s, constraints)]
        scored = [score_slip(s) for s in feasible]
        beam = diversify(top_k(scored, constraints.beam_width))
    return pareto_select(beam, constraints.output_count)

```

37.4 Deterministic reproducibility

Per ogni esecuzione devono essere persistiti input hash, feature schema, model version, calibration version, optimizer configuration, candidate universe hash e random seed quando un algoritmo stocastico è coinvolto. La stessa configurazione deve poter ricreare lo stesso risultato o spiegare quali dati sono cambiati.

38. Design system e componenti UI

38.1 Tokens

Token	Uso
bg.app	sfondo principale blu notte
bg.panel	card/pannelli
border.subtle	bordi sottili
accent.primary	azione principale e value positivo
state.warning	warning e uncertainty
state.negative	EV negativo/rischio
text.primary	numeri e titoli
text.secondary	metadati e spiegazioni

38.2 Componenti fondamentali

- AppShell: sidebar + content + context panel.
- FilterBar: input quota, periodo, competizioni, sport, pulsante analisi.
- OpportunityTable: righe virtualizzate, sorting e filtri inline.
- MetricPill: odds, probability, EV, edge, confidence.
- StatusBadge: Strong/Candidate/Watch/Neutral/Avoid/Invalid.
- LineupCard: XI, modulo, confirmed state e freshness.
- NewsTimeline: eventi ordinati per observed_at.
- ProbabilityCard: valore, intervallo, metodo, calibration state.
- MarketExplorer: tabella gerarchica market family → market type → selections.
- SlipBuilder: candidate slip interattiva con vincoli visibili.
- WhyPanel: fattori, counterfactual e model consensus.

38.3 Responsive behavior

- Desktop: layout a tre colonne come mockup.
- Tablet: sidebar collassabile e pannello laterale convertito in drawer.
- Mobile: flusso verticale; filter bar sticky; candidate slip in card list; dettagli in accordion.
- Tabelle: colonne secondarie comprimibili e apertura del dettaglio al tap.

39. User journey e stati dell'app

39.1 Percorso principale

1. Selezionare quota target, competizioni, mercati e profilo rischio.
2. Avviare l'analisi del palinsesto.
3. Vedere il riepilogo: fixture analizzate, mercati valutati, candidate disponibili.
4. Aprire una candidate slip e analizzare le singole leg.
5. Aprire il dettaglio di una fixture per verificare statistiche, lineup, news e mercati.
6. Confrontare varianti della slip e motivazioni.
7. Salvare una candidate slip in paper mode.
8. Valutare il risultato nella sezione storico/backtest.

39.2 Stati UI per una fixture

Stato	Visualizzazione	Azione consentita
FRESH	dati aggiornati	analisi completa
STALE	badge e avviso	richiedi refresh
LINEUP_PENDING	formazione attesa	watch
LINEUP_CONFIRMED	badge confermata	ricalcolo
SOURCE_DEGRADED	fonte parzialmente indisponibile	analisi ridotta
MARKET_SUSPENDED	mercato non prezzabile	attesa riapertura
FINISHED	fixture conclusa	review/backtest

40. Contratti dati e schema JSON

40.1 Opportunity object

```
{
  "fixture_id": "...",
  "market_code": "TOTAL_GOALS",
  "selection": {"line": 2.5, "side": "OVER"},
  "odds": {"value": 1.85, "source": "...", "observed_at": "..."},
  "probability": {"raw": 0.608, "calibrated": 0.623},
  "fair_odds": 1.605,
  "edge": 0.099,
  "ev": 0.152,
  "uncertainty": 0.18,
  "data_quality": 0.94,
  "status": "STRONG",
  "model_version": "...",
  "calibration_version": "..."
}
```

40.2 Candidate slip object

```
{
  "constraints": {...},
  "selections": [...],
  "aggregate": {
    "odds": 7.42,
    "joint_probability": 0.286,
    "fair_odds": 3.50,
    "ev": 1.123,
    "uncertainty": 0.27,
    "correlation_penalty": 0.08
  },
  "explanation": {...}
}
```

40.3 Evidence object

```
{
  "source_id": "...",
  "source_level": "A",
  "observed_at": "...",
  "published_at": "...",
  "entity_ids": [...],
  "event_type": "LINEUP_CONFIRMED",
  "confidence": 0.99,
  "raw_ref": "...",
  "structured_payload": {...}
}
```

41. Model lifecycle e retraining

41.1 Ciclo del modello

```
DATASET BUILD → TEMPORAL SPLIT → TRAIN → CALIBRATE → VALIDATE
      ↓
METRICS ← DRIFT MONITOR ← DEPLOYED MODEL ← MODEL CARD
```

41.2 Model registry

- model_id
- sport
- market_family
- feature_schema_version
- training_cutoff
- artifact_hash
- calibration_id
- validation_metrics
- status: candidate/active/retired
- owner/evidence

41.3 Promozione modello

La promozione deve dipendere da test automatici di regressione e calibrazione. Un modello nuovo non può diventare active soltanto perché migliora una metrica su un singolo split; deve mantenere qualità accettabile sui segmenti principali e superare controlli di leakage.

41.4 Champion/challenger

Il sistema può mantenere un modello active e uno o più challenger. Le predizioni dei challenger possono essere calcolate in shadow mode per confrontarne calibrazione, stabilità e market behavior prima di una sostituzione.

42. Failure modes e graceful degradation

Failure mode	Rilevazione	Comportamento
Provider odds down	health check/retry	usa ultimo snapshot con badge stale o blocca EV
Lineup unavailable	freshness check	mantieni expected lineup con confidence ridotta
News parser failure	schema validation	conserva raw item e non aggiorna feature
Provider disagreement	consistency check	abbassa data quality e segnala mismatch
Model artifact missing	registry lookup	blocca market family interessata
Optimizer timeout	job watchdog	restituisce best feasible candidate parziale con stato esplicito
DB latency	metrics/timeout	usa cache per letture non critiche
Invalid market schema	validation	isola solo quel market type

43. Scenario di utilizzo end-to-end

L'utente apre AlgoWinBet, seleziona un insieme di competizioni e un intervallo di quota, lascia attivi i mercati supportati e avvia l'analisi. Il sistema costruisce l'universo delle fixture, scarica e normalizza il catalogo dei mercati, recupera quote e dati sportivi, applica il cutoff informativo, produce feature, esegue i modelli, calibra le probabilità e calcola fair odds, edge ed EV.

Le opportunità vengono poi sottoposte a controllo qualità, uncertainty filtering e correlation analysis.

L'optimizer costruisce molte combinazioni candidate, applica i vincoli hard, ordina le candidate secondo l'obiettivo configurato e restituisce un piccolo insieme diversificato. La dashboard mostra subito la candidate principale, i suoi valori aggregati e le motivazioni.

Quando arriva una nuova informazione rilevante, il sistema non riparte necessariamente da zero: identifica i mercati sensibili, aggiorna le feature interessate, ricalcola le sole previsioni impattate, aggiorna le correlazioni e, se necessario, rigenera le candidate slip. La timeline mantiene ogni stato precedente.

43.1 Esperienza percepita dall'utente

- “Ho il palinsesto completo davanti.”
- “Vedo dove il modello rileva una discrepanza tra prezzo e probabilità.”
- “So quanto è robusta la stima.”
- “Posso capire quali informazioni hanno modificato la previsione.”
- “Posso esplorare una partita e tutti i suoi mercati.”
- “Posso costruire una combinazione senza controllare manualmente centinaia di opzioni.”
- “Posso verificare a posteriori se il sistema era calibrato.”

44. Principi di implementazione per Codex

44.1 Ordine di costruzione del codice

1. Creare il dominio canonico e gli schema Pydantic.
2. Creare il layer database e migration.

3. Creare provider adapters mock e reali.
4. Implementare normalization e provenance.
5. Implementare fixture state builder e snapshot.
6. Implementare market registry e market schemas.
7. Implementare baseline probabilistiche.
8. Implementare feature engine.
9. Implementare ML models e calibration.
10. Implementare pricing/value engine.
11. Implementare correlation engine.
12. Implementare optimizer.
13. Implementare news/lineup engine.
14. Implementare event bus e selective recalculation.
15. Implementare backtesting e paper betting.
16. Implementare API.
17. Implementare dashboard e match detail.
18. Implementare model/data monitoring.
19. Integrare tutti i test e il percorso end-to-end.

44.2 Contratto di ogni modulo

Ogni package deve dichiarare input, output, error types, dipendenze, metriche e test. Nessun modulo deve leggere direttamente dati grezzi da un provider diverso dal proprio adapter.

44.3 Regola di sviluppo

Prima si costruisce una versione deterministica e osservabile del flusso. Gli elementi stocastici devono essere introdotti con seed, metriche e controlli. Ogni incremento deve mantenere una modalità mock che consenta di testare l'intera pipeline senza dipendere da una sorgente esterna.

45. Appendice: configurazioni e parametri

45.1 Configurazione logica

```
app:
  supported_sports: [...]
  enabled_market_families: [...]
  default_language: "it"
data:
  provider_priority: [...]
  snapshot_retention: "configurable"
model:
  ensemble_weights: "learned"
  calibration_method: "configured per market family"
optimizer:
  beam_width: "configurable"
  output_count: "configurable"
risk:
  max_exposure_per_fixture: "configurable"
  max_exposure_total: "configurable"
ui:
  theme: "dark"
  default_view: "dashboard"
```

45.2 Regola per nuove market family

1. Definire MarketType e schema dei parametri.
2. Definire la funzione di normalizzazione e la semantica degli outcome.
3. Definire il dataset e le feature necessarie.
4. Scegliere baseline e modello specializzato.
5. Definire calibration strategy e metriche.
6. Definire pricing/fair odds.
7. Definire correlation rules.
8. Definire rendering UI.
9. Aggiungere test sintetici e backtest segmentato.
10. Abilitare il market type nel registry soltanto dopo il superamento dei controlli.

Definizione finale di AlgoWinBet

AlgoWinBet deve essere percepito come un sistema di analisi e ottimizzazione fondato su dati, probabilità calibrate, prezzi di mercato, informazione temporale e gestione dell'incertezza. La qualità del prodotto deriva dalla capacità di spiegare, misurare e riprodurre ogni decisione tanto quanto dalla qualità della previsione.